

Ferramenta Inteligente para Comunicação Proativa com o Segurado

Monitorar clima, identificar risco, decidir quem avisar e escrever o aviso — antes do sinistro. Abaixo, as 31 tarefas que fecham 100% do enunciado, divididas em cinco frentes que rodam em paralelo.

PRAZO	DIAS RESTANTES	TAREFAS	FRENTES PARALELAS
13/09/2026, 23h59	23	31	5

Plano derivado do enunciado oficial (páginas 16 a 20 de *Desafios 3, 4, 5 e Projeto final*). Fonte de dados, detecção de raio, limiares e nível de congelamento verificados por chamada real à API.

01 Antes de dividir: três coisas que mudam

O Desafio 5 não é o 4 com outro tema. Três diferenças alteram o planejamento.

O GitHub público deixou de ser opcional

O enunciado se contradiz: em *Entregáveis* o repositório aparece como opcional, mas em *Observações importantes* lê-se “o repositório do GitHub **deverá** possuir acesso público” e que o README precisa conter instruções de instalação e execução, além de informar a licença MIT.

A única leitura em que as duas frases coexistem sem risco é fazer o repositório. Trate como obrigatório — e crie-o na primeira semana, não na última.

A base de segurados não vem pronta

No Desafio 4 os CSVs de notas fiscais vinham do curso. Aqui não há base alguma: o grupo precisa **criar** os segurados fictícios, com apólice, cidade e coordenadas. Isso é pré-requisito das regras de negócio e da geração de mensagens — sem ele, duas frentes ficam paradas.

Muito do Desafio 4 é reaproveitável

Copie e adapte em vez de reescrever: `app/config.py` (multi-provedor de LLM e leitura de chaves), o tratamento de limite de requisições do `app/agent.py`, `scripts/gerar_pdf.py` e `scripts/gerar_entrega.py` — este último já audita credenciais antes de empacotar. São várias horas economizadas nas frentes C e E.

02 Decisões técnicas já validadas

Testei as opções de fonte de dados hoje para o grupo não gastar tempo descobrindo isso na marra.

Use a Open-Meteo: sem chave, sem cadastro, funciona no Brasil

Testada com coordenadas de São Paulo: retornou 72 horas de previsão com precipitação (mm), rajadas de vento (km/h), código WMO de tempo e CAPE (J/kg). Não exige chave de API — uma credencial a menos para gerenciar e esconder. O enunciado aceita “outras fontes públicas, desde que devidamente documentadas”.

```
https://api.open-meteo.com/v1/forecast
?latitude=-23.55&longitude=-46.63
&hourly=precipitation,wind_gusts_10m,weather_code,cape
&timezone=America/Sao_Paulo&forecast_days=3
```

Raio é detectável no Brasil; o campo dedicado a ele, não

A variável `lightning_potential` é aceita pela API mas volta **vazia** para o Brasil — o mesmo problema dos códigos WMO 96 e 99 (trovoada com granizo), preenchidos só na Europa Central.

O **código WMO 95** (trovoada), esse sim, funciona: em Manaus ele apareceu junto de CAPE de 3.450 J/kg. Detectar raio é portanto código 95 mais CAPE acima do limiar — dois campos que a API entrega para o Brasil. A troca de granizo por raio, decidida pelo grupo, saiu mais barata do que o cenário original.

03 Cenários decididos pelo grupo

Duas apólices e quatro eventos, fechados na Fase 0. Sete das oito combinações entram no motor de regras — a oitava é descartada por razão de mérito, explicada abaixo.

EVENTO	RESIDENCIAL	AUTOMOTIVA	SINAL NA API
Chuva intensa	Alagamento, infiltração, telhado	Aquaplanagem, veículo em via alagada	<code>precipitation</code>
Raio	Surto elétrico, queima de aparelhos	— não se aplica	<code>weather_code 95 + cape</code>
Vento forte	Telhas, antenas, objetos soltos	Queda de árvore sobre o veículo	<code>wind_gusts_10m</code>
Granizo	Telhado, claraboias, vidros	Lataria e para-brisa	<code>cape + freezing_level_height</code>

Raio × automotiva é o par a descartar

Um automóvel é uma gaiola de Faraday: durante uma tempestade elétrica ele é um dos lugares mais seguros onde se pode estar. Não há recomendação preventiva honesta a dar ao segurado — e uma mensagem do tipo “cuidado com raios no seu carro” é tecnicamente incorreta, o oposto do que o desafio avalia em qualidade das mensagens. Ficam **sete cenários ativos**.

Granizo: o proxy correto é o nível de congelamento, não o CAPE

CAPE alto explica por que a tempestade se forma, mas não se a pedra chega ao chão — ela derrete na queda. O que decide isso é a `freezing_level_height`: quanto mais baixa a altitude onde a atmosfera atinge 0 °C, menor o percurso derretendo. A variável está disponível para o Brasil.

Medido hoje, o gradiente confirma a física e a climatologia do país: no Sul o nível de congelamento cai a **1.760 m** em Porto Alegre e 2.370 m em Caxias do Sul, enquanto no Norte fica entre 4.370 e 5.320 m. É exatamente onde o granizo é frequente no Brasil. O contraste mais útil é Manaus — CAPE de 3.450 J/kg, dos mais altos do país, mas nível de congelamento a 4.740 m: muita descarga elétrica, granizo raro. Distinguir raio de granizo por essa variável separa os dois eventos com fundamento, em vez de arbitrar dois limiares no mesmo sinal.

Limiares de partida para a tarefa B.2, medidos hoje em 15 capitais: chuva intensa a partir de 10 mm/h ou 40 mm em 48 h; vento forte a partir de 60 km/h de rajada; raio com código WMO 95 e CAPE de 800 J/kg; granizo com trovoada, CAPE acima de 1.500 J/kg e nível de congelamento abaixo de 3.800 m. São ponto de partida, não conclusão — calibrar e justificar é a tarefa B.4.

04 Fase 0 — Fundação

Concluída. As quatro decisões saíram e estão no repositório: a matriz de cenários, os contratos de dados em `app/schemas.py`, o repositório público com licença MIT e o provedor de LLM em `app/config.py`. As cinco frentes estão livres para começar.

FASE 0

Contratos e alicerce

Dias 1-2 · concluída

F0.1

Escolher os cenários de negócio

Decidido: duas apólices (residencial e automotiva) e quatro eventos (chuva intensa, raio, vento forte e granizo) — matriz na seção 03. As 15 cidades monitoradas estão em `data/cidades.csv`.

CONCLUÍDA

duas apólices, quatro eventos e sete cenários ativos, fixados como constante em `app/schemas.py`.

F0.2

Congelar os contratos de dados

Definir os três formatos que atravessam o sistema: `Segurado`, `EventoClimatico` e `Notificacao` — nomes de campos, tipos e unidades. É isso que permite cinco pessoas programarem ao mesmo tempo.

CONCLUÍDA

`app/schemas.py`, com 12 testes que também servem de exemplo de uso para as frentes.

F0.3

Criar o repositório público

Repositório no GitHub com acesso público, `LICENSE MIT` na raiz, estrutura de pastas, `requirements.txt`, `.gitignore` com `.env` e `.env.example`.

CONCLUÍDA

repositório público, licença MIT reconhecida e `main` protegida exigindo PR.

F0.4

Definir provedor de LLM e chave

Reaproveitar o `config.py` do Desafio 4. O Gemini já está validado e tem camada gratuita; cada integrante usa a própria chave localmente, no `.env`.

CONCLUÍDA

`app/config.py` com três provedores; cada integrante usa a própria chave no `.env`.

05 Fase 1 — Cinco frentes em paralelo

Letras, não números: estas frentes não têm ordem entre si. Todas dependem apenas da Fase 0 e podem avançar simultaneamente.

FRENTE A

Coleta meteorológica

Dias 3-10

Atende ao requisito “consumir dados de pelo menos uma API pública de informações meteorológicas”.

A.1

Cliente da Open-Meteo

Função que recebe latitude e longitude e devolve a previsão horária das variáveis escolhidas. Sem dependência de framework — `urllib` ou `requests` resolvem.

PRONTO QUANDO

uma chamada por coordenada devolver a série temporal já tipada.

A.2

Cache e tolerância a falha

Cache local em disco por cidade e hora, com timeout, retry e mensagem clara quando a API estiver fora. Evita repetir chamadas idênticas durante a demonstração.

PRONTO QUANDO

a segunda chamada para a mesma cidade não tocar a rede, e a queda da API não derrubar a aplicação.

A.3

Normalizador de resposta

Converter o JSON bruto no contrato `EventoClimatico` definido em F0.2, com unidades explícitas e fuso horário do Brasil.

depende de F0.2

PRONTO QUANDO

as frentes B e C conseguirem consumir a saída sem conhecer a API de origem.

A.4

Segunda fonte: alertas do INMET

Opcional, mas rende ponto no critério “correta integração com fontes externas”. Serve para cruzar a previsão calculada com alertas oficiais já emitidos.

PRONTO QUANDO

os alertas oficiais aparecerem ao lado da previsão — ou quando o grupo registrar por que descartou a fonte.

FRENTE B

Segurados e regras de negócio

Dias 3–12

Atende a “identificar automaticamente eventos climáticos relevantes” e “aplicar regras de decisão para determinar quais segurados devem receber notificações”. É o coração avaliado em “clareza das regras de negócio”.

B.1

Base sintética de segurados

CSV com algo entre 30 e 100 segurados fictícios: nome, tipo de apólice, cidade, UF, latitude, longitude e canal preferido. Cobrir capitais de regiões climáticas diferentes para os cenários aparecerem.

PRONTO QUANDO

cada cenário de F0.1 tiver ao menos um segurado elegível na base.

B.2

Classificador de eventos

Traduzir números em eventos nomeados com severidade: chuva intensa, raio, vento forte e granizo. Chuva e vento saem direto de precipitação e rajada; raio de código WMO 95 mais CAPE; granizo acrescenta o nível de congelamento.

PRONTO QUANDO

a mesma previsão sempre produzir a mesma classificação, e cada limiar tiver um número justificado.

B.3

Motor de regras

Regras em arquivo de configuração, não no código: evento × tipo de apólice × severidade decide se notifica e com que urgência. Deixar as regras legíveis por quem não programa é o que se avalia.

PRONTO QUANDO

for possível alterar um limiar sem tocar em nenhum arquivo `.py`.

B.4

Justificar cada limiar

Documentar de onde vem cada número — classificação do INMET, literatura, critério do grupo. Este texto vira direto uma seção do relatório.

PRONTO QUANDO

nenhum limiar do sistema for um número sem procedência declarada.

FRENTE C

Agentes e geração de mensagens

Dias 5–14 · Nicole + Juliana

Atende a “gerar mensagens personalizadas utilizando IA ou modelos de linguagem”. A dica do enunciado sugere quatro agentes especializados — coletor, analista, regras e redator.

C.1

Estrutura dos agentes e orquestrador

Um módulo por agente, mais um orquestrador que encadeia coleta → análise → decisão → redação. Cada agente com uma responsabilidade só.

depende de F0.2

PRONTO QUANDO

o orquestrador rodar de ponta a ponta com agentes ainda vazios, devolvendo dados de mentira.

C.2

Agente redator com LLM

Prompt que recebe o segurado, o evento e a regra acionada, e escreve a mensagem no tom do canal. Variar por perfil: residencial, automóvel, empresarial.

PRONTO QUANDO

o mesmo evento gerar mensagens visivelmente diferentes para perfis diferentes.

C.3

Guardrails da mensagem

Impedir que o LLM invente números — só pode citar dados vindos da previsão. Respeitar limite por canal (SMS em 160 caracteres) e proibir promessa de cobertura, que é risco regulatório.

PRONTO QUANDO

uma bateria de mensagens geradas passar na conferência sem número inventado nem estouro de limite.

C.4

Fallback sem LLM

Modelo de mensagem por template para quando a API do LLM falhar ou a cota estourar. Vale ouro no dia da apresentação.

PRONTO QUANDO

a demonstração completa rodar com a chave de API removida.

FRENTE D

Simulação de envio e demonstração

Dias 6–15

Atende a “simular o envio das notificações” e “permitir demonstrar o fluxo completo da solução”.

D.1

Caixa de saída simulada

Registrar cada notificação com destinatário, canal, evento, mensagem, timestamp e status. Nada é enviado de verdade — o enunciado dispensa explicitamente o envio real.

PRONTO QUANDO

uma execução deixar um registro auditável de tudo que “sairia”.

D.2

Modo cenário forçado

Injetar condições climáticas extremas sem depender do tempo real. Sem isso, um dia de céu limpo na apresentação significa nenhum evento e nenhuma mensagem para mostrar.

PRONTO QUANDO

qualquer cenário de F0.1 puder ser disparado sob encomenda.

D.3

Interface de demonstração

Streamlit mostrando as quatro etapas em sequência: previsão obtida, eventos detectados, segurados selecionados e mensagens geradas. O fluxo tem que ser visível, não inferido.

PRONTO QUANDO

alguém de fora do grupo entender o encadeamento só olhando a tela.

D.4

Demonstração por linha de comando

Script que executa tudo de ponta a ponta e imprime o resultado — serve de teste de integração e gera as evidências que vão para o relatório.

PRONTO QUANDO

um comando produzir o conjunto de mensagens de exemplo pronto para colar.

FRENTE E

Documentação e entrega

Dias 3–20 · contínuo

Começa junto com as outras frentes, não no fim. Cobre os três entregáveis e o critério “organização da documentação”.

E.1

README exigido pelo enunciado

Instruções de instalação e execução, e menção explícita à licença MIT. São requisitos nomeados nas observações importantes — não é o README genérico.

PRONTO QUANDO

alguém de fora conseguir instalar e rodar seguindo só o README.

E.2

Diagrama de arquitetura

O caminho do dado, da API até a mensagem, passando pelos quatro agentes. Se for ASCII, usar apenas caracteres simples — caracteres de caixa somem na conversão para PDF, como aconteceu no Desafio 4.

PRONTO QUANDO

o diagrama sobreviver à exportação em PDF sem perder bordas.

E.3

Relatório técnico

As cinco seções pedidas: arquitetura, descrição dos agentes, tecnologias utilizadas, fluxo de processamento e exemplos de mensagens geradas.

consume saídas de B.4 e D.4

PRONTO QUANDO

cada uma das cinco seções existir e nenhuma estiver marcada como pendente.

E.4

Galeria de mensagens por cenário

O enunciado pede “demonstrar diferentes tipos de mensagens para diferentes cenários”. Reunir um exemplo real por cenário de F0.1, com o evento que o disparou ao lado.

PRONTO QUANDO

houver uma mensagem gerada por cenário, cada uma com sua origem rastreável.

06 Fase 2 — Integração

O momento em que as frentes se encontram. Reserve tempo real para isso: é onde aparecem as divergências de contrato que ninguém previu.

FASE 2

Costura e endurecimento

Dias 16–19

I.1

Fluxo real ponta a ponta

Substituir todos os dados de mentira pelos módulos reais e rodar o caminho completo: API → evento → regra → mensagem → caixa de saída.

PRONTO QUANDO

uma execução limpa gerar notificações a partir de previsão real, sem intervenção manual.

I.2

Bateria de cenários

Rodar todos os cenários de F0.1 e conferir manualmente se cada decisão de notificar faz sentido. Verificar também o caso “nenhum evento” — silêncio também é resposta correta.

PRONTO QUANDO

cada cenário tiver sido conferido por alguém que não escreveu a regra.

I.3

Revisão de segurança

Confirmar que nenhuma chave está no código, no histórico do Git ou no ZIP. Atenção ao histórico: um `.env` commitado por engano continua lá mesmo depois de apagado.

PRONTO QUANDO

uma busca por padrões de chave no repositório e no pacote não retornar nada.

07 Fase 3 — Entrega

FASE 3

Empacotamento e envio

Dias 20–22 · nunca no dia 23

Z.1

Gerar o PDF do relatório

Reaproveitar o `scripts/gerar_pdf.py` do Desafio 4. Conferir o resultado renderizado, não só o texto extraído.

PRONTO QUANDO

o PDF estiver revisado página por página, sem sobras de rascunho.

Z.2

Gerar o ZIP do código

Reaproveitar o `scripts/gerar_entrega.py`, que já exclui ambiente virtual e cache e aborta se encontrar credencial.

PRONTO QUANDO

o ZIP extraído em pasta limpa rodar sem depender de nada externo.

Z.3

Conferir o repositório

Abrir o link do GitHub numa janela anônima para provar que o acesso é realmente público, com README e LICENSE visíveis.

PRONTO QUANDO

o repositório abrir sem login.

Z.4

Enviar

PDF, ZIP e link do repositório, enviados pela representante do grupo. Depois, revogar as chaves de API que tiverem circulado.

PRONTO QUANDO

houver confirmação de recebimento.

08 Rastreabilidade: requisito → tarefa

Cada exigência do enunciado com a tarefa que a cumpre. Use como lista de conferência final — se toda linha estiver coberta, a entrega está completa.

EXIGÊNCIA DO ENUNCIADO	TAREFAS
Consumir pelo menos uma API pública de meteorologia	A.1 · A.2 · A.4
Identificar automaticamente eventos climáticos relevantes	A.3 · B.2
Aplicar regras de decisão sobre quem notificar	B.1 · B.3 · B.4
Gerar mensagens personalizadas com IA ou LLM	C.2 · C.3 · C.4
Simular o envio das notificações	D.1
Demonstrar o fluxo completo da solução	D.2 · D.3 · D.4 · I.1
Relatório: arquitetura da solução	E.2 · E.3
Relatório: descrição dos agentes	C.1 · E.3
Relatório: tecnologias utilizadas	E.3
Relatório: fluxo de processamento	E.2 · E.3
Relatório: exemplos das mensagens geradas	D.4 · E.4
ZIP com todo o código-fonte	Z.2
Repositório GitHub com acesso público	F0.3 · Z.3
README com instalação, execução e licença MIT	F0.3 · E.1
Ocultar chaves de API e credenciais	F0.3 · F0.4 · I.3
Separar coleta, processamento, decisão e comunicação	C.1 · F0.2
Agentes especializados por responsabilidade	C.1
Diferentes mensagens para diferentes cenários	C.2 · E.4 · I.2

09 Divisão sugerida

Uma frente por pessoa. A Juliana aparece duas vezes: as tarefas dela se concentram no fim do cronograma, então até o dia 14 ela reforça a frente mais pesada.

INTEGRANTE	FRENTE	TAREFAS	POR QUÊ
Daniel Ramon	A — Coleta meteorológica	A.1–A.4	Já conhece o código do Desafio 4, que dá a base de configuração e cache.
Paulo Henrique	B — Segurados e regras	B.1–B.4	Trabalha com seguros: define quais eventos importam para cada apólice e sustenta a justificativa das regras no relatório.
Nicole Paes	C — Agentes e mensagens	C.1–C.4	Tem mais experiência com LLM, e esta frente concentra o item de maior peso no critério de uso de IA.
Juliana Catarina	C — apoio, até o dia 14	C.2 · C.3	A frente mais pesada recebe reforço enquanto a documentação ainda não começou. E a tarefa E.4 dela é literalmente a saída de C.2.
Paulo Roberto	D — Simulação e demonstração	D.1–D.4	Dono da apresentação: o modo cenário forçado é o que garante a demo.
Juliana Catarina	E — Documentação e entrega	E.1–E.4 · Z.1–Z.4	Representante do grupo, portanto responsável natural pelo envio final. Concentra-se a partir do dia 15.

A Fase 2 é dos cinco: integração feita por uma pessoa só é integração que esconde problema. A Fase 0 já está concluída.